

ShieldLabs Security Report

test-repoo-shieldlabs

81 Good	Scan Type	CODE
	Scan ID	scan_html_test1
	Generated	2026-07-01 09:24 UTC
	Total Findings	3
	Attack Chains	1

Severity Breakdown

CRITICAL		1 (33.3%)
HIGH		1 (33.3%)
MEDIUM		1 (33.3%)
LOW		0 (0.0%)
INFO		0 (0.0%)

Critical & High-Risk Findings - Immediate Attention Required

CRITICAL	
SQL Injection	
app.py:4	
HIGH	
Hardcoded Secret	
app.py:10	

Findings Summary

Severity	Vulnerability	Location	CVSS
CRITICAL	SQL Injection	app.py:4	9.3
HIGH	Hardcoded Secret	app.py:10	8.1
MEDIUM	Weak Hashing	auth.py:4	5.8

Attack Chains

CRITICAL - Time to Exploit: 2 hours

SQL injection combined with an exposed database port allows direct DB access, bypassing the application layer entirely.

Impact: Full database compromise.

Detailed Findings

1. SQL Injection

File: app.py:4 | CVSS: 9.3 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:L)

Vulnerable code:

```
query = "SELECT * FROM users WHERE name = '" + username + "'"
```

Suggested fix:

```
query = "SELECT * FROM users WHERE name = %s"
```

AI Review: The proposed fix is safe and complete.

Business Impact: Complete data loss or theft.

2. Hardcoded Secret

File: app.py:10 | CVSS: 8.1 ()

Vulnerable code:

```
API_KEY = "sk-live-51H8xJ2eZvKYlo2C0FAKEKEYFORTESTING"
```

Suggested fix:

```
API_KEY = os.getenv('SHIELDLABS_API_KEY')
```

3. Weak Hashing

File: auth.py:4 | CVSS: 5.8 (CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:N/A:N)

Vulnerable code:

```
return hashlib.md5(password.encode()).hexdigest()
```

Suggested fix:

```
return hashlib.sha256(password.encode()).hexdigest()
```